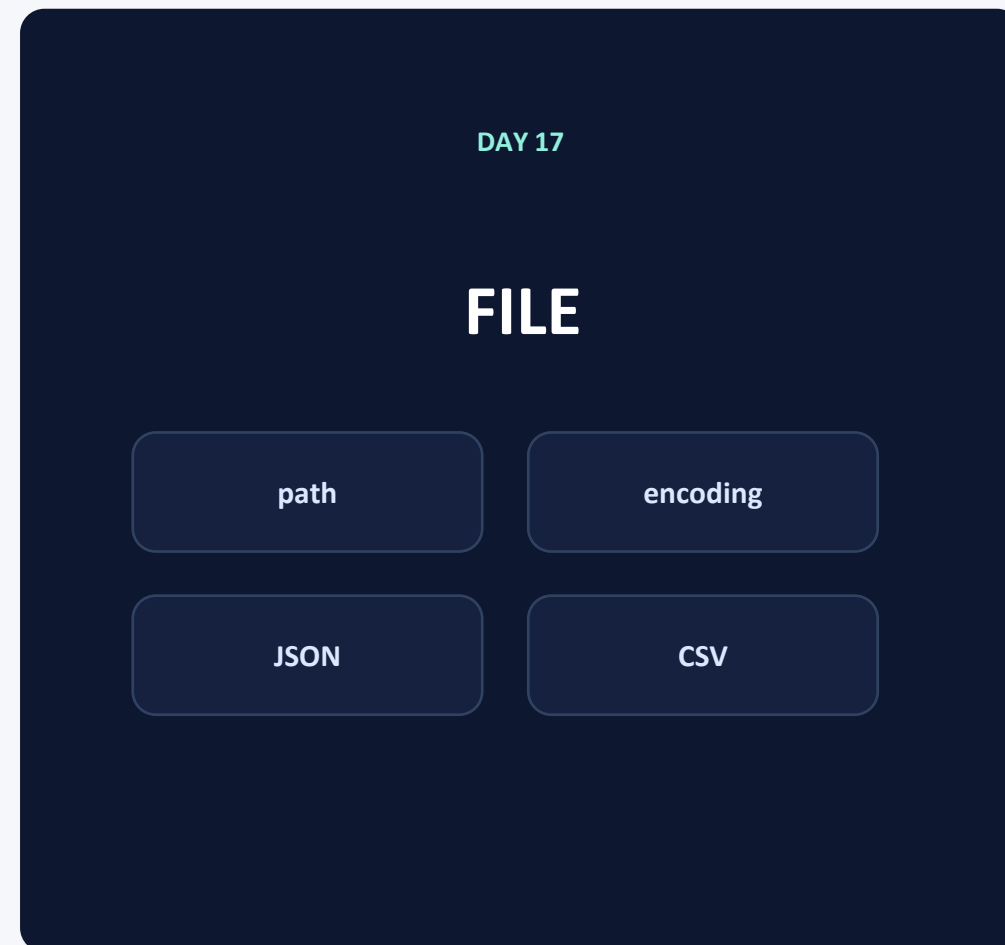


Файлы, JSON и CSV

Читать и записывать файлы, работать с путями и кодировкой, сохранять структурированные данные в JSON и табличные — в CSV.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Читать и записывать файлы, работать с путями и кодировкой, сохранять структурированные данные в JSON и табличные — в CSV.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Модули, пакеты и зависимости

Что уже есть в нашей модели.



СЕГОДНЯ

Файлы, JSON и CSV

Вчера: Модули, пакеты и зависимости. Сегодня мы добавляем следующий слой: файлы, json и csv.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Программе часто нужно пережить перезапуск и обменяться данными с другими системами. Файлы — самый простой мост к постоянному хранению.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Файл — контейнер на диске; JSON — подписанная структура данных, CSV — таблица строк и колонок с разделителями.

ПЕРЕВОД В IT

file

именованная область долговременного хранения данных

path

адрес файла или папки в файловой системе

encoding

правило преобразования символов в байты и обратно

Слой 1: file • path • encoding

Сначала смысл, затем профессиональный термин.

**file**

именованная область долговременного хранения данных

**path**

адрес файла или папки в файловой системе

**encoding**

правило преобразования символов в байты и обратно

Слой 2: JSON • CSV • serialization

Сначала смысл, затем профессиональный термин.

**JSON**

текстовый формат структурированных данных

**CSV**

текстовый табличный формат с разделителями

**serialization**

преобразование объекта в формат хранения или передачи

Слой 3: read mode • write mode

Сначала смысл, затем профессиональный термин.

**read mode**

режим открытия файла для чтения

**write mode**

режим открытия файла для записи

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: parser • delimiter

Сначала смысл, затем профессиональный термин.

parser

программа, разбирающая текстовый формат

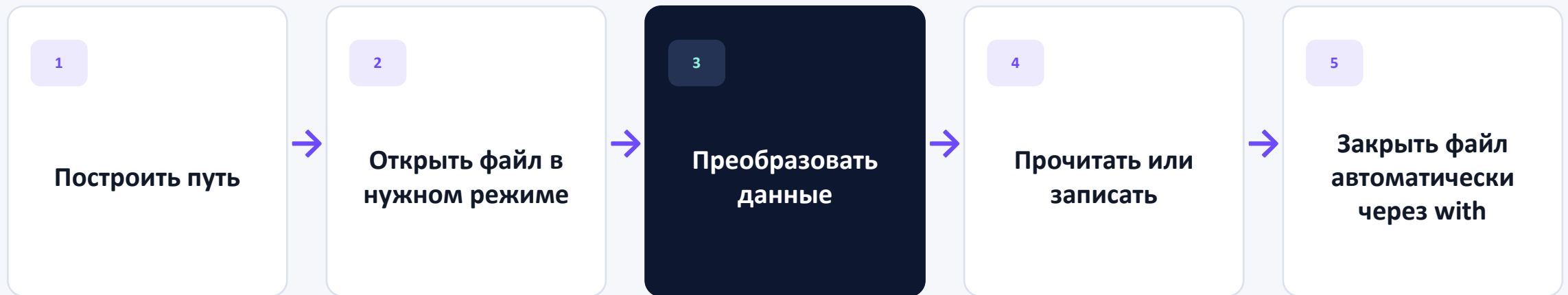
delimiter

разделитель полей в табличном текстовом файле

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day17_step1

```
import json

profile = {"name": "Anna", "age": 25}
with open("profile.json", "w", encoding="utf-8") as file:
    json.dump(profile, file, ensure_ascii=False)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
import json
```

Python выполняет эту строку сверху вниз.

```
profile = {"name": "Anna", "age": 25}
```

Сохраняем значение или результат вычисления под понятным именем.

```
with open("profile.json", "w",  
as file:
```

Сохраняем значение или результат вычисления под понятным именем.

```
json.dump(profile, file,
```

Сохраняем значение или результат вычисления под понятным именем.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
import json

profile = {"name": "Anna", "age": 25}
with open("profile.json", "w", encoding="utf-8") as file:
    json.dump(profile, file, ensure_ascii=False)
```

РЕЗУЛЬТАТ

Файл profile.json создан с объектом name/age

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
import json
with open("profile.json", encoding="utf-8") as file:
    profile = json.load(file)
print(profile["name"])
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Anna

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

```
import json  
data = json.load("profile.json")
```

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

`json.load()` ожидает файловый объект, а не строку с именем файла. Сначала файл нужно открыть.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.

```
import json
with open("profile.json", encoding="utf-8") as file:
    data = json.load(file)
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Сохранить список задач в `tasks.json`, затем прочитать его обратно.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Создаём Python-список словарей.**
- 2 Открываем файл на запись.
- 3 `json.dump` сериализует данные.
- 4 Открываем файл на чтение.
- 5 `json.load` восстанавливает Python-структуру.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
import json
tasks = [{"title": "Учиться", "done": False}]
with open("tasks.json", "w", encoding="utf-8") as f:
    json.dump(tasks, f, ensure_ascii=False)
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
import json
tasks = [{"title": "Учиться", "done": False}]
with open("tasks.json", "w", encoding="utf-8") as f:
    json.dump(tasks, f, ensure_ascii=False)
with open("tasks.json", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded[0]["title"])
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

УЧИТЬСЯ

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Запиши CSV с колонками name,score и двумя строками данных.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Модуль csv и DictWriter помогут явно задать названия колонок.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
import csv
rows = [{"name": "Anna", "score": 90}, {"name": "Max", "score": 80}]
with open("scores.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "score"])
    writer.writeheader()
    writer.writerows(rows)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

scores.csv: заголовок name,score и две строки

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

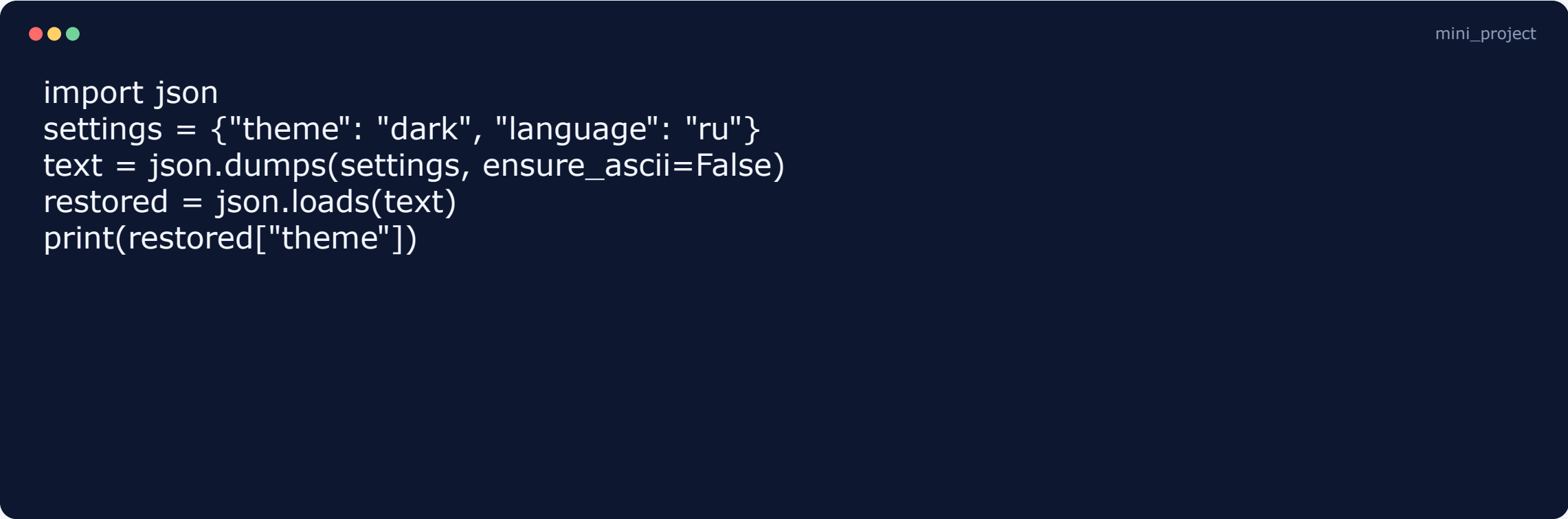
ЗАДАНИЕ

Сохрани настройки приложения в JSON и прочитай значение theme.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.

A dark-themed terminal window with a title bar at the top containing three colored circles (red, yellow, green) on the left and the text 'mini_project' on the right. The terminal contains a Python script that imports the json module, creates a settings dictionary, dumps it to a string, loads it back, and prints the 'theme' value.

```
import json
settings = {"theme": "dark", "language": "ru"}
text = json.dumps(settings, ensure_ascii=False)
restored = json.loads(text)
print(restored["theme"])
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

dark

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 Забывать `encoding="utf-8"`.

02 Путать `json.load` и `json.loads`.

03 Записывать в файл без `with` и не закрывать его.

04 Считать CSV полноценной базой данных.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «file» и какую проблему это понятие помогает описать?

2

Чем «file» отличается от «JSON»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 file — именованная область долговременного хранения данных.

2 file: именованная область долговременного хранения данных. JSON: текстовый формат структурированных данных.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **Файл profile.json создан с объектом name/age**

4 **json.load() ожидает файловый объект, а не строку с именем файла. Сначала файл нужно открыть.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Читать и записывать файлы, работать с путями и кодировкой, сохранять структурированные данные в JSON и табличные — в CSV.
- ✓ Читать и объяснять пример темы «Файлы, JSON и CSV».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Объектно-ориентированное программирование

Завтра: Объектно-ориентированное программирование. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.